

 Docs

- Mobile monitoring >
- Model performance monitoring >
- Queues and Streams >
- Network monitoring >
- OpenTelemetry >
- SAP Solutions >
- Serverless monitoring >
- Streaming Video & Ads >
- Synthetic monitoring >
- Website performance monitoring >

DATA INSIGHTS

- New Relic AI >
- Alerts >
- Change tracking >
- Charts, dashboards, and querying >
- Cloud Cost Intelligence >
- CodeStream >
- Distributed tracing >
- Errors inbox >
- Workloads >
- New Relic Lens >

NRQL

- Get started with NRQL
- How to query with NRQL
- NRQL charts and dashboards
- NRQL syntax**
- NRQL tutorials
- NRQL querying
- NRQL examples
- Schedule NRQL searches

- Service Architecture Intelligence >
- Service level management >
- Workflow Automation >

SECURITY

NRQL reference

To write good [NRQL](#) queries, it helps to understand how our various NRQL clauses and functions work. This doc contains definitions of NRQL clauses and functions, and gives examples of how to use them.

Looking for basic NRQL syntax rules? See [How to use NRQL](#). For a tutorial, see [Introductory NRQL tutorial](#).

Query components

As noted in our [basic NRQL syntax doc](#), every NRQL query will contain a `SELECT` clause and a `FROM` clause, with all other clauses being optional. The clause definitions below also contain example NRQL queries.

DELETE keyword

The `DELETE` keyword in New Relic Query Language (NRQL) is used to remove specific data and attributes within Pipeline Control. This helps manage the data that is ingested into the New Relic Database (NRDB).

The `DELETE` keyword is applied in NRQL queries to specify which data or attributes should be removed. Here are some examples:

Here're some examples:

- Deleting specific metrics:

```
DELETE FROM Metric
WHERE metricName =
'newrelic.goldenmetrics.infra.kubernetes'
```



 Copy

- Removes metrics with the specified name.

- Deleting specific spans:

On this page

- Query components
- Query metric data
- Functions
- Type conversion
- Comments
- Related docs

Was this doc helpful?

 Yes  No

 Edit this doc

```
DELETE FROM Span WHERE appName =  
'My Application'
```

Copy

- Removes spans associated with the specified application.

• **Deleting specific agent updates:**

```
DELETE FROM AgentUpdate WHERE a =  
'b'
```

Copy

- Removes agent updates where the attribute `a` equals `b`.

Required clauses

Required: SELECT statement

Required: FROM clause

Optional clauses

AS clause

COMPARE WITH clause

EXTRAPOLATE clause

FACET clause

FACET ... AS clause

FACET CASES clause

FACET ... ORDER BY clause

JOIN clause

LIMIT clause

OFFSET clause

ORDER BY clause

PREDICT clause

SHOW EVENT TYPES clause

SINCE clause

SLIDE BY clause	▼
TIMESERIES clause	▼
UNTIL clause	▼
WHERE clause	▼
WITH METRIC_FORMAT clause	▼
WITH ... AS clause (NRQL variables)	▼
WITH TIMEZONE clause	▼

Query metric data

Metric data is more complex than other types of data. There are specific tips for querying it well. We have two types of metric data, each with their own query guidelines:

- [Query dimensional metrics](#), which are reported by our Metric API and by some of our solutions that use that API (for example, our [Dropwizard integration](#) or [Micrometer integration](#)).
- [Query metric timeslice data](#), which is our original metric data type reported by our APM, mobile monitoring, and [browser monitoring](#) ⓘ.

For more details about how we report metric data, see [Metric data types](#).

Functions

In this section we explain NRQL functions, both [aggregator functions](#) and [non-aggregator functions](#).

Aggregator functions

You can use aggregator functions to filter and aggregate data. Some tips for using these:

- If you're using an aggregator function multiple times in the same query (for example, `SELECT`

median(one_metric),
median(another_metric)), it can
cause problems in displaying
results. To solve this, use the [AS
function](#). For example:

```
SELECT median(one_metric) AS 'med-  
a', median(another_metric) AS 'med-  
b'
```

Copy

- Data type "coercion" is not supported. Read about [available type conversion functions](#).
- For how to display results over time, see [Group results over time](#).

Examples:

```
SELECT histogram(duration, 10, 20) FROM  
PageView SINCE 1 week ago
```

Copy

aggregationendtime()	▼
apdex(attribute, t:)	▼
average(attribute)	▼
bucketPercentile(attribute)	▼
cardinality([metric_name, include:{attribute_list}, exclude:{attribute_list}])	▼
cdfPercentage(attribute, threshold [, threshold [, ...]])	▼
count(*)	▼
derivative(attribute [,time interval])	▼
earliest(attribute)	▼
filter(function(attribute), WHERE condition)	▼
funnel(attribute, steps)	▼
histogram(attribute[, width: n][, buckets: n][, start: n])	▼
keyset()	▼

<code>latest(attribute)</code>	▼
<code>latestrate(attribute, time interval)</code>	▼
<code>max(attribute)</code>	▼
<code>median(attribute)</code>	▼
<code>min(attribute)</code>	▼
<code>percentage(function(attribute), WHERE condition)</code>	▼
<code>percentile(attribute [, percentile [, ...]])</code>	▼
<code>predictLinear(attribute, [,time interval])</code>	▼
<code>rate(function(attribute) [,time interval])</code>	▼
<code>stdvar(attribute)</code>	▼
<code>sum(attribute)</code>	▼
<code>uniqueCount(attribute, [, attribute [, ...]] [, precision: number])</code>	▼
<code>uniques(attribute [,limit])</code>	▼

Non-aggregator functions

Use non-aggregator functions to return values for each data point within NRQL queries.

<code>accountId()</code>	▼
<code>aparse(attribute, pattern)</code>	▼
<code>blob(attribute)</code>	▼
<code>buckets(attribute, ceiling [,number of buckets])</code>	▼
<code>concat(attribute, [, attribute [, ...]] [, precision:])</code>	▼
<code>convert(attribute, fromUnits, toUnits)</code>	▼
<code>capture(attribute, regular expression)</code>	▼
<code>decode(input, encoding)</code>	▼

<code>dimensions(include: {attributes}, exclude: {attributes})</code>	▼
<code>encode(input, encoding)</code>	▼
<code>cidrAddress(attribute [, number [, cidrFormat])</code>	▼
<code>eventType()</code>	▼
<code>getField(attribute, field)</code>	▼
<code>getCdfCount(attribute, threshold)</code>	▼
<code>if(condition, trueValue [, falseValue])</code>	▼
<code>jparse(attribute [, path])</code>	▼
<code>length(attribute)</code>	▼
<code>contains(attribute, element)</code>	▼
<code>lookup(table)</code>	▼
<code>lower(str)</code>	▼
<code>mapKeys(attribute)</code>	▼
<code>mapValues(attribute)</code>	▼
<code>minuteOf(attribute), hourOf(attribute), etc.</code>	▼
<code>mod(attribute, divisor)</code>	▼
<code>position(str, substr [, occurrence])</code>	▼
<code>round(attribute)</code>	▼
<code>stddev(attribute)</code>	▼
<code>abs(attribute)</code>	▼
<code>floor(attribute)</code>	▼
<code>ceil(attribute)</code>	▼
<code>clamp_max(attribute, limit)</code>	▼
<code>clamp_min(attribute, limit)</code>	▼
<code>pow(attribute, exponent)</code>	▼
<code>sqrt(attribute)</code>	▼
<code>exp(attribute)</code>	▼

<code>ln(attribute)</code>	▼
<code>log2(attribute)</code>	▼
<code>log10(attribute)</code>	▼
<code>log(attribute, base)</code>	▼
<code>string(attribute [, precision:])</code>	▼
<code>substring(str, start [, length])</code>	▼
<code>toDatetime(timestamp[, pattern [, timezone]])</code>	▼
<code>toTimestamp(datestring[, pattern [, timezone]])</code>	▼
<code>upper(str)</code>	▼

Type conversion

NRQL does not support "coercion."

This means that a float stored as a string is treated as a string and cannot be operated on by functions expecting float values.

You can convert a string with a numeric value or a boolean with a string value to their numeric and boolean equivalents, or convert a non-string value to a string value, with these functions:

- Use the `numeric()` function to convert a number with a string format to a numeric value. The function can be built into a query that uses math functions on query results or NRQL aggregator functions, such as `average()`. Please note that if the NRQL value is in the [gauge format](#), then `numeric()` won't work on it. Instead, you must use one of these compatible query functions:
 - `latest()`
 - `min()`
 - `max()`

- `sum()`
- `count()`
- `average()`
- Use the `boolean()` function to convert a string value of `"true"` or `"false"` to the corresponding boolean value.
- Use the `string()` function to convert a numeric, boolean, tuple, or array value to a string value. See [string\(\)](#) above for more information.

Comments

When writing a NRQL query, you can add comments, which can help your team members better understand and use the query.

Here are syntax details:

- `--` Two dashes will comment out all text to the right of this indicator on the same line.
- `//` Two slashes will comment out all text to the right of this indicator on the same line.
- `/* */` Any text in between these character sets will be commented out. This indicator can apply to multiple lines.

Note that comments aren't displayed everywhere. Some views, like "recent queries" and "view query," won't show comments.

Some example queries that include comments:

```
FROM Transaction SELECT
uniqueCount(appId) -- This will return
the number of unique App IDs
```

 Copy

```
FROM TransactionError
SELECT count(*) SINCE 1 day ago //
Transaction Error for the past day
```

 Copy


```
FROM TransactionTrace /* This data may  
be incomplete;  
If so, run a query of Transaction */  
SELECT count(*)
```

 Copy

Related docs

Other popular resources for understanding NRQL syntax and rules include:

- [NRQL instructional course](#) 
from New Relic University
- [Learn how to query the Metric data type](#)
- [Use subqueries](#)
- [Use funnels to evaluate a series of related data](#)

[Careers](#) [Terms of Service](#) [DMCA Policy](#) [Privacy](#) [Website Terms](#)
[Your Privacy Choices](#)  [Patent Notice](#) [UK Slavery Act](#)

COPYRIGHT © 2026 NEW RELIC INC. This site is protected by reCAPTCHA and the Google [Privacy Policy](#) and [Terms of Service](#)